

CHIZUBA

技術解説と想定質疑

千葉県地図系サービスを 1 つに束ね、住民と行政が相互に情報を投稿できるウェブサイト。本書は、このアプリを**発表者自身が説明できる**ようにするための技術解説である。構成・技術選定の理由・各機能の内部の動き・想定される質問への答えを、実際のコードから起こしてまとめている。

チーム	愛で千葉は救えるのか
プロダクト	CHIZUBA (CHiba + ZU (地図) + BA (場))
対応範囲	千葉県全域 (デモデータは市川市・JIS X 0402 コード 12203)
起動	<code>cd app && docker compose up</code> → <code>http://localhost:3000</code>
コンテナ	2 つ (<code>node:22-slim</code> / <code>postgres:17-alpine</code> . いずれも Docker 公式イメージ)
外部サービス	国土地理院タイル / 重ねるハザードマップ / OSRM / 気象庁 JSON (すべて認証キー不要)
発表	2026-09-16 (水) 11:50 提出・発表 10 分 + 質疑 3 分

発表前に読むなら、この順で

時間が無いときは **6 章 (想定質疑)** と **巻末の早見表** だけでよい。質疑で数字を聞かれたら早見表、仕組みを聞かれたら 5 章の該当節、「なぜその技術か」を聞かれたら 4 章の該当行を思い出せばよい形にしてある。2 章と 3 章は、全体像が頭に入っていないと 6 章の答えが暗記になってしまうので、余裕があれば先に通しておく。

この資料は `main` (コミット 5dbf879) 時点のコードを読んで書いている。図の正本は同じディレクトリの `chizuba-overview.progfocus.md` (prog-focus 形式)。

目次

1 CHIZUBA は何をするアプリか	3
一言でいうと	3
機能の一覧 (F-1 ~ F-8)	3
誰が使えて、どこからログインが要るか	3
2 全体アーキテクチャ	4
動いているもの：コンテナ 2 つと外部サービス 4 種	4
コードの地図：6 つのかたまり	5
「1 箇所を直せば全体が追従する」ようにしてある	5
3 リクエストの流れ (3 本)	6
3-1 地図を開く (GET /)	6
3-2 浸水を投稿する (POST /api/reports)	6
3-3 ログインする (デモモードの場合)	7
4 技術選定の「なぜ」	8
全体を貫く 1 つの制約	8
なぜ Next.js なのか	8
なぜ Google マップではなく MapLibre なのか	8
なぜ PostgreSQL で、なぜ PostGIS を使わないのか	9
なぜ静的な GeoJSON を DB に入れないのか	9
なぜこの Docker 構成なのか	9
なぜ Tailscale Funnel なのか (提出物ではない)	9
5 各機能の仕組み	11
5-1 ハザードマップの重ね (F-1)	11
5-2 3 種類の投稿を 1 つのモデルに統一した (F-2 / F-3 / F-6)	11
5-3 雨量の自動記録 (F-3)	12
5-4 注意案内と、越えてはいけない線 (F-4)	12
5-5 徒歩ナビ (F-5)	13
5-6 景観スポットと写真 (F-5)	13
5-7 行政の応答 (F-7)	13
5-8 認証の二段構え (F-8)	13
5-9 セッションを、このデータベースに縛る	14
5-10 行政ロールの PIN (公開運用のときだけ)	14
5-11 公開 URL をリクエストから決める	15
5-12 書き出し (CSV / GeoJSON)	15
5-13 入力 of 健全性	15
5-14 デモデータの扱い	16
6 想定質疑と模範回答	17
仕組みと技術選定について	17
データと権利について	18
運用と実用性について	18
7 用語集	20
ウェブとサーバー	20
認証とセキュリティ	20
地図とデータ	20
Docker とデータベース	21
気象と法令	21
8 数字の早見表	22
データの件数	22
構成と規模	22
上限としきい値	22
外部サービスとの約束	22
覚えておく固有の値	23

CHIZUBA は何をするアプリか

一言でいうと

千葉県オープンデータは、公開された時点で止まっている。避難場所も AED も景観スポットも「市が持っている静的な一覧」であって、**現場で今どうなっているか**——ガードレールが折れている、この道路が冠水している、この時期はここが綺麗——は誰も更新できない。

CHIZUBA は、そのオープンデータを土台に置いたうえで、**その上に住民の投稿を重ね、行政が応答できる場所を作る**。地図はすでに共通言語になっているので、防災と観光という性格の違う 2 つの用途を、同じ地図・同じ投稿の仕組みで扱える。

質疑で最初に返すべき 1 文

「行政が出したオープンデータの上に、住民が投稿を重ねて、行政がそれに応答する。その一往復を 1 枚の地図の上で完結させたのが CHIZUBA です。」

機能の一覧 (F-1 ~ F-8)

要件はチームで 8 つに決めてある。ここに無いものは作らず、ここにあるものは全部作った。

ID	機能	中身
F-1	ハザードマップ表示	洪水・高潮・津波の浸水想定を地図に重ねる。レイヤーごとに ON/OFF と不透明度を変えられ、凡例と出典を出す
F-2	危険箇所の市民報告	壊れたガードレール・陥没した路面などを、位置 + 写真 + 説明で投稿する
F-3	浸水状況の報告	冠水している場所を投稿する。投稿時点の雨量をサーバーが自動で記録する
F-4	注意案内	過去に浸水報告のある地域に、雨の予報が出ているとき注意を出す
F-5	観光マップ	景観 100 選を地図に載せ、日英の解説・写真・徒歩ルートを出す
F-6	観光おススメの投稿	「ここが綺麗」「この店の〇〇が良い」を F-2 と同じ操作感で投稿する
F-7	行政からの応答	行政が公式コメントを付け、対応状況を 4 段階で更新する
F-8	Google ログイン	Google OAuth。鍵が未設定の環境では自動でデモログインに落ちる

これに加えて、設計に落とす過程で 5 つだけ足した。対応状況の 4 段階 (F-7 の中)、投稿一覧の画面、投稿日の範囲での絞り込み、CSV / GeoJSON の書き出し、キーワード検索である。どれも「なぜ足したか」と「削るときに何を消せばよいか」を要件定義に書いてあるので、勝手に膨らんだ機能は 1 つも無い。

誰が使えて、どこからログインが要るか

権限は 3 段階。未ログインでも「見る」ことは全部できる。防災情報をログインの壁の向こうに置かないという判断で、投稿にだけログインを要求している。

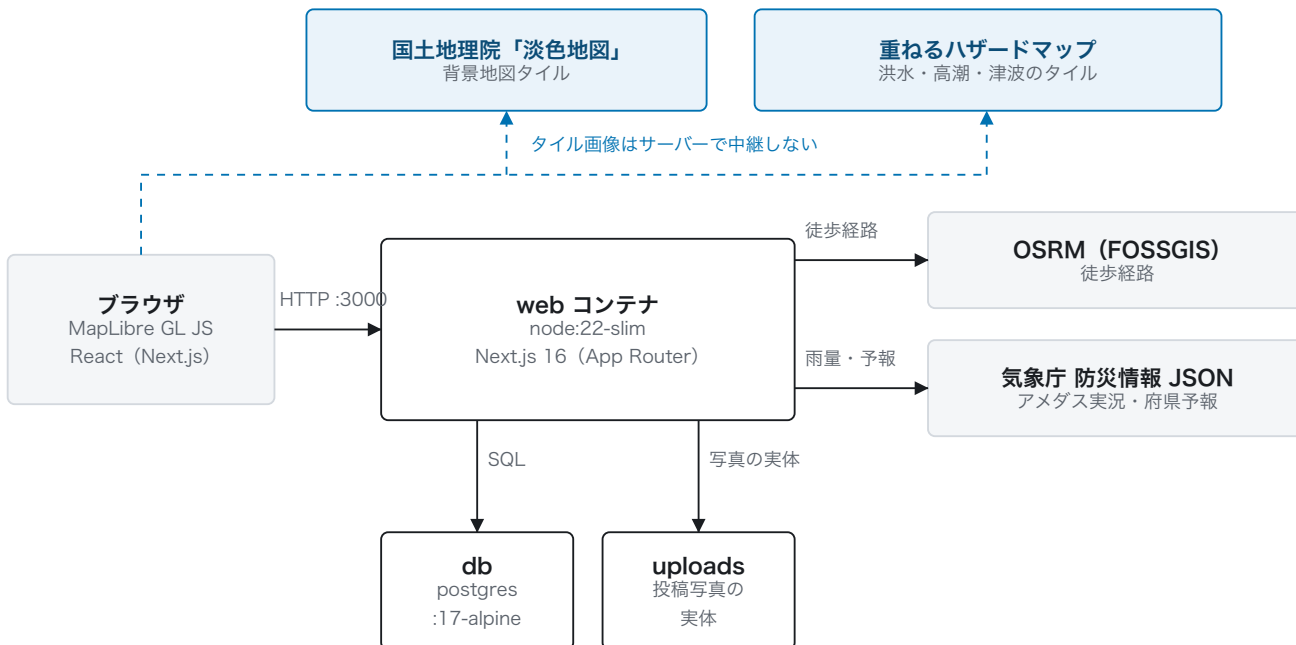
ロール	誰か	できること
未ログイン	誰でも	地図・ハザードマップ・全レイヤー・投稿の閲覧、徒歩経路、注意案内、書き出し
一般ユーザー	住民・来訪者	+ 投稿 (危険箇所・浸水・観光おススメ)、自分の投稿の編集と削除、他人の投稿へのコメント
行政ユーザー	自治体職員	+ 公式コメント (区別表示)、対応状況の更新、公式おススメ投稿

行政ユーザーは一般ユーザーの上位互換で、専用の管理画面は作っていない。同じ地図画面の上で出せる操作が増えるだけにしている。画面数を増やさずに実装量を抑えるためで、そのぶん「行政の発言と住民の発言が混ざる」危険が出るので、行政の投稿とコメントには必ずバッジを出し、地図ではピンの輪を公式色にしている。

全体アーキテクチャ

動いているもの：コンテナ 2 つと外部サービス 4 種

図 2-1 実行時の構成。太い矢印がサーバーを通る経路、破線がブラウザから直接取りに行く経路



静的なオープンデータ（避難場所・AED・子育て施設・景観 100 選）は GeoJSON として `app/public/data/` に同梱しており、起動時にどこへも取りに行かない。

読み方の要点は 4 つある。

① コンテナは 2 つだけ

web (Next.js) と db (PostgreSQL 17)。どちらも Docker 公式イメージで、タグを固定している。db はホストにポートを公開していない。公開すると審査員のマシンで 5432 番が埋まっていたときに起動しなくなるため、中を覗きたいときは `docker compose exec db psql` で入る。

② タイル画像だけはブラウザが直接取りに行く

背景地図とハザードマップは画像タイルなので、サーバーで中継しても意味がなく、むしろ web が詰まる。一方 徒歩経路と気象データは必ずサーバーを通す。理由は 3 つで、利用規約が求める User-Agent を確実に付けること、レート制限 (OSRM は 1 秒 1 リクエスト) をサーバー側 1 箇所ですること、タイムアウトを入れて外部が落ちても画面が固まらないようにすること。

③ 起動の順番は healthcheck で保証する

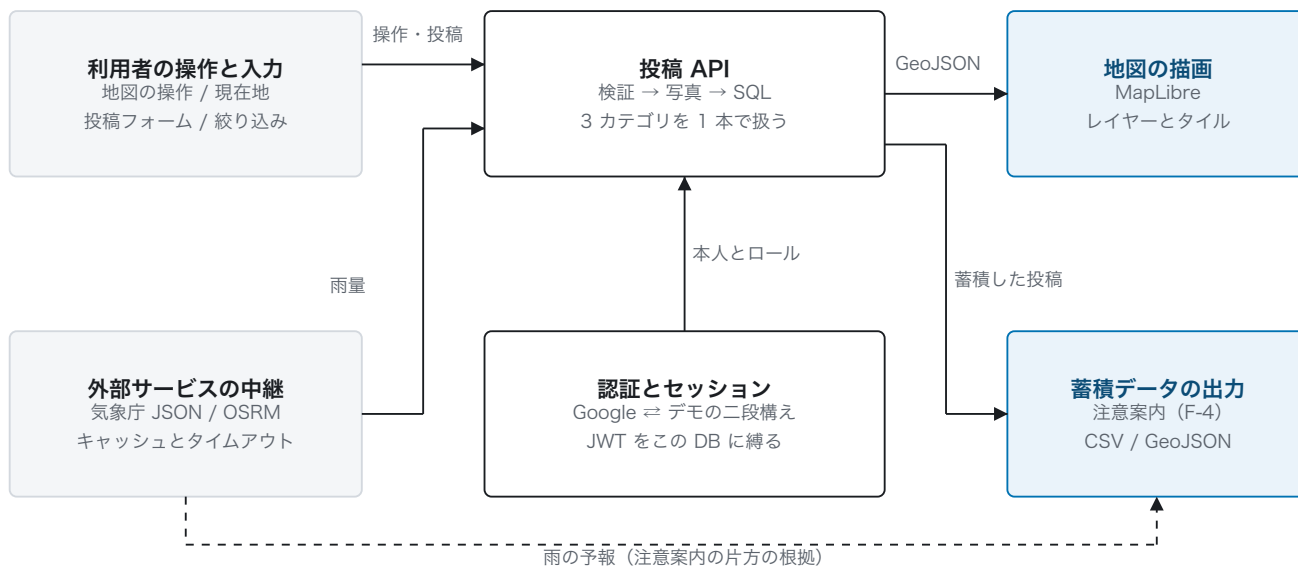
db に `pg_isready` の healthcheck を付け、web の `depends_on` に `condition: service_healthy` を書いてある。これで DB が受け付けられる状態になるまで web は起動しない。配布資料「Docker コンテナの作成例」の例 2 と同じ作法である。

④ 外部が落ちても地図と投稿は出る

施設の点は同梱の GeoJSON、投稿は DB にあるので、OSRM も気象庁も落ちた状態で表示できる。経路は直線距離からの見積もりに切り替わり、見積もりであることを画面に出す。浸水の投稿も、雨量が取れなくても投稿自体は成功させる。現場で投稿できないほうが害が大きいという判断である。

コードの地図：6 つかたまり

図 2-2 コードを 6 つかたまりで見たところ。詳細は同梱の prog-focus 図（ノード 38・接続 34）



かたまりごとの実体は次のとおり。どのファイルを開けばよいかまで覚えておくと、「そこはどう実装したのか」と聞かれたときに詰まらない。

かたまり	何をしているか	中心のファイル
利用者の操作と入力	画面の状態を 1 つ持ち、地図・操作パネル・投稿パネルへ配る	<code>components/MapExplorer.tsx</code>
投稿 API	検証 → 写真の保存 → SQL の 3 段。3 カテゴリを 1 テーブルで扱う	<code>app/api/reports/</code>
地図の描画	MapLibre の実体を持つ唯一の場所。モードを変えても作り直さない	<code>components/MapView.tsx</code>
認証とセッション	Google とデモの分岐。JWT をこのインストールに縛る	<code>lib/auth.ts</code>
外部サービスの中継	気象庁 JSON の取得とキャッシュ、OSRM の中継	<code>lib/jma.ts</code>
蓄積データの出力	注意案内の判定、CSV / GeoJSON の組み立て	<code>lib/weather.ts</code> ・ <code>lib/reportExport.ts</code>

「1 箇所を直せば全体が追従する」ようにしてある

質疑で拡張性を聞かれたときに、この 3 つを挙げられるとよい。

増やすもの	やること
投稿のカテゴリを増やす	<code>lib/reports.ts</code> の <code>REPORT_CATEGORIES</code> に 1 行足す。フォームの入力欄・地図のピンの色・詳細パネル・書き出しの列が、すべてこの表を読んでいるので自動で増える
対応する市町村を増やす	<code>municipalities</code> テーブルに 1 行足し、その市の CSV を GeoJSON にする。初期表示の座標もズームも範囲もマスタから引いているので、コードは 1 行も変えない
地図に載せるデータを増やす	<code>lib/layers.ts</code> の <code>LAYERS</code> に 1 行足す。色・アイコン・ポップアップの項目も同じ行に書く

ここは正直に言う

デモデータは市川市だけである。「千葉県全域に対応」と言えるのは骨格がそうになっているからであって、54 市町村ぶんのデータを入れてあるわけではない。聞かれたら「全域対応の設計にしてあり、データは市川市を入れています。増やす作業はマスタに 1 行と GeoJSON 化だけです」と答える。

リクエストの流れ（3 本）

ここは「動かしたときに何が順番に起きるか」を追う章である。質疑で「投稿すると何が起きるんですか」と聞かれたら、3-2 の 13 行を思い出せばよい。

3-1 地図を開く（GET /）

1	ブラウザ	GET /?mode=disaster を開く
2	web	サーバーコンポーネントが getSessionView() を呼ぶ。認証モードとログイン状態はサーバー側で 1 回だけ決め、画面へ渡す
3	web	HTML を返す。地図の状態を持つクライアントコンポーネントに、ログイン状態・市町村コード・初期モードを渡す
4	ブラウザ	MapLibre を初期化する。背景タイル（国土地理院「淡色地図」）はブラウザが直接取りに行く
5	ブラウザ	モードの初期値から、どのレイヤーを出すかを定める。防災モードなら避難場所・AED・子育て施設 + ハザードマップ、観光モードなら景観 100 選
6	外部	ハザードタイルをブラウザが直接取得する。ズーム 6 より引いた縮尺では取りに行かない（読み取れない染みのために国交省へ投げ続けないため）
7	web	GET /data/*.geojson。同梱の静的ファイルで、DB を経由しない
8	ブラウザ	GET /api/reports?city=12203&... を呼ぶ。ログインは要らない
9	db	矩形（地図の表示範囲）・カテゴリ・期間・キーワードで絞る SQL を 1 本流す
10	web	GeoJSON の FeatureCollection で返す。静的レイヤーと同じ形なので、地図側のコードが 1 本で済む
11	ブラウザ	GET /api/weather?city=12203 を呼ぶ
12	外部	web が気象庁 JSON を取りに行く。キャッシュに載っていれば行かない（実況 10 分・予報 30 分）
13	ブラウザ	注意案内（F-4）を出すか決める。根拠になる浸水報告だけは期間の絞り込みを無視して全期間で引き直す

13 行目がなぜ必要か

期間で絞ったせいで「過去に浸水報告はありません」と表示されると、防災の判断を誤らせる。絞り込みは見せ方の都合であって、危険の有無は絞り込みで変わってはいけない。

3-2 浸水を投稿する（POST /api/reports）

1	ブラウザ	地図で位置を決め、カテゴリ「浸水」・冠水の深さ・本文・写真（最大 3 枚）を入れる
2	ブラウザ	POST /api/reports に multipart/form-data で送る
3	web	ログインしていなければ 401。閲覧は自由だが書き込みは要ログイン
4	web	content-length を先に見て、10 MB を超えるなら本文を読み込まずに 413 を返す
5	web	入力を検証する。写真は JPEG / PNG / WebP ・1 枚 5 MB ・3 枚まで。申告された MIME を信じず、先頭バイトでも形式を確かめる
6	web	座標を市町村マスタの範囲と突き合わせて city_code を決める。クライアントの申告は使わない
7	外部	最寄りのアメダスを探す。距離の近い順に見て、1 時間降水量が正常な値で入っている最初の観測所を採る（20 km 以内）
8	web	取れたら雨量・観測時刻・観測所名・距離を投稿に焼き込む。取れなくても投稿は通す（現場で投稿できないほうが害が大きい）
9	web	写真の実体を uploads ボリュームへ保存する。ファイル名はサーバーが作る
10	db	トランザクションで reports に 1 行 + report_photos に n 行
11	web	途中で失敗したら ROLLBACK し、保存済みの写真の実体も消す（孤児ファイルを残さない）

- | | | |
|----|------|------------------------|
| 12 | web | 201 と、作られた投稿の内容を返す |
| 13 | ブラウザ | 地図の投稿ソースを引き直し、新しいピンを描く |

この 13 行のうち、覚えておくと強い 3 つ

6 行目：所属市町村をサーバーが決めるので、他市の投稿として紛れ込ませられない。7～8 行目：雨量はクライアントから受け取らない。F-4 の注意案内の根拠になる値なので、投稿者が自由に入れられてはいけない。11 行目：DB とファイルは別の場所なので、片方だけ成功した状態を残さないよう明示的に後始末している。

3-3 ログインする（デモモードの場合）

- | | | |
|----|------|---|
| 1 | ブラウザ | GET /login を開く |
| 2 | web | 起動時に決まっている認証モードを見る。デモモードなら表示名の入力欄、Google モードなら「Google でログイン」 |
| 3 | ブラウザ | 表示名とロール（一般／行政）を送る。公開運用の環境では、行政を選んだときだけ PIN も送る |
| 4 | web | PIN を確かめる。未設定の環境では素通りする（審査員の体験を変えない） |
| 5 | db | users に upsert する。同じ「表示名 × ロール」で入り直したら同じユーザーになる |
| 6 | db | app_instance.install_id を読む。このデータベースの識別子 |
| 7 | web | 署名鍵をその ID から導く。sha256("chizuba-session-key:" + install_id)。鍵を 1 つも設定していない環境でも、鍵は環境ごとに違う |
| 8 | web | JWT に uid / 表示名 / ロール / 担当市町村 / 発行元のインストール ID を載せて署名する |
| 9 | web | Cookie を配ってリダイレクトする。飛び先は設定値ではなく、リクエストが名乗ったホストから毎回導く |
| 10 | web | 以降のリクエストでは、トークンの中の ID と今の ID を毎回突き合わせる。違えば未ログイン扱い |

Google モードとの違いは 3 つだけである。/login に出るボタンが Google になること、コールバックが /api/auth/callback/google を通ること、そしてロールを自己申告できないこと（環境変数 GOV_ACCOUNTS に書いたメールアドレスにだけ行政ロールが付く）。5 行目以降——ユーザーの upsert、インストール ID の照合、リダイレクト先の導出——はまったく同じコードを通る。二段構えといっても、分岐しているのは入り口だけである。

技術選定の「なぜ」

全体を貫く 1 つの制約

認証キーが要る外部サービスを使わない

審査員が `docker compose up` だけで全機能を試せることを最優先にした。コード評価は **実装した割合 × 正しく動作する割合** の掛け算なので、「鍵が無いと動かない機能」は、実装した分がまるごと 0 点になる。唯一の例外が Google OAuth で、これは要件に明記されているため外せない。だから **鍵が未設定なら自動でデモログインに落ちる** 構成にした。

この制約が、以下の選定のほぼすべてを決めている。地図・ハザードマップ・徒歩経路・気象データはいずれも「無料で使える似たサービス」ではなく、**登録もキーも要らないもの**を選んでいく。

領域	採ったもの	なぜ
背景地図	国土地理院「淡色地図」	無認証。出典の明記だけで使える。国内の地名表記が正確で、配信も安定している
地図ライブラリ	MapLibre GL JS	無認証。純 JavaScript でネイティブ拡張を持たないので、Docker のビルドが壊れない
ハザードマップ	重ねるハザードマップのラスタスタイル	無認証。ポリゴンを DB に入れずに済み、実装が最小になる
徒歩経路	OSRM (FOSSGIS e.V.)	無認証で徒歩プロファイルがある。実測で疎通を確認済み
気象データ	気象庁 防災情報 JSON	無認証。一次情報なので出典として強い
フレームワーク	Next.js (App Router)	地図の UI とサーバー側の中継 API を 1 コンテナに収められる
データベース	PostgreSQL 17 (postgres:17-alpine)	Docker 公式イメージ。住民と行政の投稿を保存する
DB クライアント	pg (node-postgres)	純 JavaScript。ORM もマイグレーションツールも入れない
CSS	Tailwind CSS v4	提出までの期間が短く、書き足しの速度を優先した
アイコン	lucide-react	依存ゼロの SVG コンポーネント。絵文字は UI に使わない
配色	Okabe-Ito	色覚多様性に配慮した配色として国内で実績がある

なぜ Next.js なのか

このアプリは **地図を描くフロントエンド**と、**外部サービスを中継するサーバー**の両方を必要とする。中継が要るのは、OSRM の利用規約が User-Agent の明示と 1 秒 1 リクエストの制限を求めている、気象庁 JSON もキャッシュしないと閲覧数に比例して取りに行ってしまうからである。

フロントエンドとサーバーを別々に立てると **コンテナが 1 つ増える**。Next.js の App Router なら、同じプロジェクトの中に画面 (React) とサーバーの API (ルートハンドラ) を並べて置き、1 つのコンテナで動く。提出要件が「`docker compose up` 一発」を求めている以上、構成部品は少ないほどよい。

実行イメージは `output: "standalone"` で作った最小構成だけを入れている。`node_modules` を丸ごと持たないので、イメージが小さく、起動も速い。

なぜ Google マップではなく MapLibre なのか

理由は 3 つある。第 1 に、Google Maps Platform は API キーと課金アカウントが必須で、上の制約に真正面からぶつかる。審査員の環境で地図が出ない。第 2 に、このアプリの主役は **国土地理院のタイル**と**国土交通省のハザードタイル**で、これらを重ねられることが要件である。MapLibre は任意の XYZ タイルをそのままレイヤーと

して重ねられるが、Google マップは自前のタイルの上に 他機関のラスタを自由に重ねる用途には向いていない。第 3 に、MapLibre は BSD ライセンスのオープンソースで、**利用規約の変更で急に使えなくなる心配がない**。

なぜ PostgreSQL で、なぜ PostGIS を使わないのか

住民と行政の投稿は **消えては困るデータ**なので、ブラウザの保存領域やファイルではなくデータベースに置く。PostgreSQL を選んだのは Docker 公式イメージがあり、タグを固定できるからである。

PostGIS を入れなかった理由は 3 つ

① 提出要件が「ベースイメージは可能な限り Docker 公式イメージ」と定めており、`postgis/postgis` は公式イメージではない。② 必要な地理演算は「地図の表示範囲に入る投稿を引く」(矩形の範囲検索)と「2 点間の距離」だけで、どちらも数値カラムと Haversine 式で足りる。③ 投稿は 1 市あたり多くて数百件の規模で、特別な索引が無くても問題にならない。PostGIS が要るのは「ポリゴンとの交差判定を DB でやりたい」ときだが、**ハザードマップはタイル画像として重ねるだけなので、その必要が無い**。

ORM とマイグレーションツールも入れていない。スキーマは `app/db/init/*.sql` を PostgreSQL の公式イメージが起動時に流す仕組みに任せてある。依存を 2 つ減らせるうえ、**審査員の初回起動で確実にスキーマができる**。代償は「スキーマを変えたらボリュームを作り直す必要がある」ことだが、開発期間が 2 週間強のプロジェクトでは割に合う。

なぜ静的な GeoJSON を DB に入れないのか

避難場所 123 件・AED 304 件・子育て施設 388 件・景観 100 選 100 件は、市川市が公開した **変わらないデータ**である。これらを DB に入れると、「起動したのにデータが入っていない」という事故が起きうる。初期データの投入が失敗しても、コンテナは正常に立ち上がってしまうからである。

そこで、手元で CSV から GeoJSON へ変換したものを **リポジトリにコミットして同梱**し、ブラウザが直接読む形にした。DB を使うのは **投稿だけ**である。「読むだけのデータ」と「書き込まれるデータ」を置き場所で分けた、と説明すればよい。

なぜこの Docker 構成なのか

決めたこと	理由
ベースイメージは公式 (DOI)	提出要件。 <code>node:22-slim</code> と <code>postgres:17-alpine</code>
タグを固定する (<code>latest</code> を使わない)	提出要件。 <code>latest</code> は将来別のものになり、再現しなくなる
multi-stage ビルド	依存の取得・ビルド・実行を分け、実行イメージに開発用の依存を持ち込まない
<code>depends_on</code> + <code>healthcheck</code>	配布資料の作成例 2 と同じ作法。 <code>pg_isready -U chizuba -d chizuba</code> が通るまで <code>web</code> を起動しない
<code>db</code> のポートを公開しない	審査員のマシンで 5432 番が埋まっていると起動できなくなるため
公開ポートは <code>\\${CHIZUBA_PORT:-3000}</code>	既定の 3000 は動かさず、塞がっている環境だけ逃がせる
<code>USER node</code> で動かす	<code>root</code> で動かさない。そのため <code>/app/uploads</code> を先に作って所有者を移してある

`healthcheck` に `-U` と `-d` を付けているのには理由がある。付けないと既定の `postgres` ユーザーで見に行き、**起動の途中でも通ってしまうことがある**。そうすると `web` が先に立ち上がって接続に失敗する。

なぜ Tailscale Funnel なのか (提出物ではない)

プレゼンで審査員に実物を触ってもらうため、自宅の Mac で動かしてインターネットへ公開する手順を別に用意してある。Tailscale Funnel を選んだのは、**外部のホスティングサービスに登録せず、無料アカウントと自宅の Mac だけで HTTPS の公開 URL を得られるからである**。見る側に Tailscale は要らず、普通のウェブサイトとして開ける。

図 4-1 公開したときの経路. TLS は Tailscale 側で終端し、自宅の中は素の HTTP のまま



ここで効いてくるのが5-11 節の仕組みである。Tailscale Funnel は公開ホスト名を `X-Forwarded-Host` に、TLS を終端したことを `X-Forwarded-Proto: https` に入れて転送する。CHIZUBA はそのヘッダーからリダイレクト先を毎回導いているので、公開 URL をどこにも書かずに、そのまま HTTPS のサイトとして成立する。起動と QR コードの生成をまとめたスクリプトも置いてあり、画面の無い Mac (`colima`) でも通るようにしてある。

これは提出物に含まれない

CODIHA に提出するのは `app/` 配下だけで、公開用の一式は提出アーカイブに入らない。「作ったものを触ってもらうための運用手順」であって、提出要件とは別の話である。ただし 公開すると誰でも行政ロールを名乗ってしまうので、そのときだけ行政ロールに PIN を要求する仕組みを足してある (5-10 節)。

各機能の仕組み

5-1 ハザードマップの重ね (F-1)

国土交通省「ハザードマップポータルサイト」の重ねるハザードマップが配信している **ラスタタイル**を、背景地図の上に重ねている。洪水・高潮・津波の 3 種類で、レイヤーごとに ON/OFF と不透明度を変えられる。

ポリゴンを DB に入れない、という判断

浸水想定区域を「地物」として扱うなら、ポリゴンを取り込んで PostGIS で交差判定する構成になる。しかし今回必要なのは「見て分かる」ことだけで、「この地点は何メートル浸かるか」を計算する機能は要件に無い。タイル画像を重ねるだけなら、データの取り込みも DB の拡張も要らない。実装が最小になり、出典も 1 行で済む。

実際に配信されているタイルの画素を数えて確かめたところ、**凡例が 2 種類あった**。同じ色でも表す深さが違うので、1 つの凡例にまとめると誤読する。

対象	段階	浅い側の違い
洪水浸水想定区域	6	いちばん浅い色（#f7f5a9）が「0.5 m 未満」を表す
津波浸水想定・高潮浸水想定区域	8	浅い側に 2 段増え、同じ #f7f5a9 が「0.3 ～ 0.5 m」を表す

凡例に必ず添えている注意書き

「色が付いていない場所が安全とは限りません」。想定区域が公表されていない河川・地域では、その場所のタイルが 404 になり、地図には何も描かれない。白紙を「危険なし」と誤読させないことは、このアプリで最も重要な表示上の約束である。実測でも、市川市には土砂災害警戒区域のタイルが存在しなかった（平地で該当区域が無い場合）。

配信されているズームは 2 ～ 17 だが、ズーム 6 より引いた縮尺では重ねを描かない。日本全体が画面に収まるような縮尺では想定区域が数ピクセルの染みにしかならず読み取れないのに、タイルの要求だけは発生し続けるからである。ズーム 6 なら本州全体が入るので、千葉県内を見ている限り消えることはない。

5-2 3 種類の投稿を 1 つのモデルに統一した (F-2 / F-3 / F-6)

危険箇所・浸水・観光おすすめは、1 つのテーブル・1 つの API・1 つのフォームで扱う。違いは `category` の値と、カテゴリ固有の項目を入れる `details` (JSON の列) だけである。

カテゴリ	category	固有の項目	サーバーが足すもの
危険箇所	hazard	危険の種別 (5 択)	—
浸水	flood	冠水の深さ (3 択)	投稿時点の雨量・観測時刻・観測所名・距離
観光おすすめ	spot	おすすめの種別 (4 択)	—

分けなかった理由は **実装量が 3 倍にならないこと**に尽きる。3 テーブルにすれば、API も 3 本、フォームも 3 つ、地図のレイヤーも 3 系統になり、「投稿の一覧」を出すたびに 3 本の問い合わせを束ねることになる。共通なのは「位置・写真・本文・投稿者・対応状況」で、これは全体の 9 割を占める。

そのうえで、統一しない部分も決めている。地図で最初から表示する組はモードごとに分け、防災の投稿と観光の投稿が同じ濃さで混ざらないようにしている。ピンの色もカテゴリごとに変えている。

拡張性を聞かれたら、ここを答える

カテゴリの定義は `lib/reports.ts` の 1 つの配列に集めてあり、フォームの入力欄・地図のピンの色・詳細パネルの表示・書き出しの列がすべてその配列を読んでいる。たとえば「不法投棄」というカテゴリを足すなら、配列に 1 行足すだけで、投稿の入り口も表示も書き出しも同時に増える。

5-3 雨量の自動記録 (F-3)

浸水の投稿には、投稿した瞬間の雨量をサーバーが記録する。値は気象庁のアメダス実況から取る。

- 1 **web** 投稿された座標から、全国 約 1,300 地点のアメダスを距離の近い順に並べる
- 2 **web** 1 時間降水量が正常な値に入っている最初の観測所を採る（風だけを測る観測所や欠測中の観測所を飛ばすため）
- 3 **web** 20 km より遠ければ諦める。その地点の雨量として使うには遠すぎる
- 4 **web** 雨量・観測時刻・観測所名・距離を投稿に焼き込む

実測で踏んだ落とし穴 3 つ

① 観測所一覧の緯度経度は【度、分】の配列であって、小数の度ではない。これを見落とすと、すべての距離計算が壊れる。② 観測値は【数値、品質フラグ】の 2 要素で、フラグが 0 でないものは欠測なので使ってはいけない。③ 市川市にはアメダスが無く、最寄り船橋で約 10 km ある。

③ のせいで、「この地点の雨量」と言い切ることができない。そこで観測所名と距離を必ず併記している。画面には「船橋アメダス（約 10.2 km）」のように出る。質疑で精度を突かれたら、「最寄り観測所の値であることを画面に明示しています」と答えればよい。

雨量はクライアントから受け取らない。これは F-4 の注意案内の根拠になる値なので、投稿者が自由に入れられてはいけない。実測でも、rainfallMm を混ぜて送信すると検証の段階で捨てられ、サーバーが自分で取った値に上書きされることを確認してある。

気象庁へ取りに行けなかったときは、雨量なしで投稿を通す。気象庁への到達を落とした状態で試したところ、投稿は 201 で成功し、詳細に「投稿時の雨量：取得できませんでした」と表示され、注意案内は出なかった。

5-4 注意案内と、越えてはいけない線 (F-4)

過去に浸水報告のある地域に雨の予報が出ているとき、操作パネルの先頭に注意カードを出し、地図の該当地点に輪を描く。出す条件は 3 つすべてが揃ったときだけである。

条件	揃わないとどうなるか
気象庁の予報が取れている	取れなければ 出さない。根拠のない警告になるため
今後 24 時間の降水確率が 30% 以上	雨の見込みが無いときに注意を出しても、注意の意味が薄れる
過去の浸水報告が 1 件以上ある	注意すべき地点が無いので出さない

しきい値を 30% にしたのは傘を持つかどうかの一般的な目安がここだからである。50% まで上げると雨でも注意が出ない回が増え、10% まで下げるとほぼ毎日出て意味を失う。この 30% は「注意カードを出すかどうか」の内部的な足切りにすぎず、画面には気象庁の降水確率そのものを出す。CHIZUBA が確率を作り直すことはしない。

気象業務法の線 —— ここは絶対に踏まない

気象業務法は、気象庁以外が予報を業として出すことを許可制にしている。だから CHIZUBA は「浸水するでしょう」「浸水のおそれがあります」に類する表現を、コードにも画面にも一切入れていない。

出しているのは検証できる事実 2 つだけである。① 住民から寄せられた浸水の報告が N 件ある。② 気象庁の予報では、千葉県〇〇部の今後 24 時間の降水確率は最大 X% である。そのうえで「過去の投稿と気象庁の予報を並べて示しています。浸水の予測ではありません」と明記し、予報が市町村単位ではなく予報区単位であることも書いている。

予報区は市町村単位では取れない。千葉県は「北西部・北東部・南部」の 3 区分が最小である。市区町村コードから予報区を引くために、気象庁の地域区分 JSON を 4 段（市区町村 → 二次細分 → 一次細分 → 発表官署）辿っている。この対応表は全国分が入っているので、DB にカラムを足さずに千葉県全域に効く。

「データが溜まるほど精度が上がる」は、どう言うか

将来像として語るのは構わないが、提出したコードが今できると主張してはいけない。「今は事実を 2 つ並べているだけです。投稿が溜まれば、どの地点がどの雨量で冠水したかというデータセットそのものがあるので、そこから先は行政や研究者が使える形にして渡すのが筋だと考えています」——これが、法にも実装にも誠実な答え方である。

5-5 徒歩ナビ (F-5)

避難場所・AED・子育て施設・景観スポットのどれにでも、現在地からの徒歩経路を出せる。経路計算は OSRM (FOSSGIS e.V. 提供) で、必ずサーバーを経由させている。

守っていること	どうやって
利用規約が求める User-Agent を名乗る	サーバー側で必ず付ける。ブラウザからは付け替えられない
1 秒あたり 1 リクエストを超えない	順番待ちの列を 1 本作り、前回の送信から 1.1 秒空くまで待たせる
外部が落ちてても画面を固めない	8 秒で打ち切る。失敗したら 直線距離からの見積もりに切り替える
出典を出す	地図の隅と /about の両方に、OSRM と OpenStreetMap を明記

見積もりに切り替わったときは、見積もりであることを画面にはっきり出す。「経路サービスに接続できませんでした」という理由も一緒に表示する。黙って直線距離を出すと、利用者は実際の経路だと思ってしまう。

5-6 景観スポットと写真 (F-5)

観光マップの主データは市川市「いちかわ景観 100 選」で、100 件すべてに座標・日本語解説・英語解説がある。解説は 19 ~ 88 文字と短いので、1 ~ 2 文のキャプションとして扱い、「音声ガイド級の解説」は謳っていない。元データの画像列は 3 通りの URL すべてで 404 だったので使えなかった。そこで写真はウィキメディア・コモンズから、再利用が許されているもの (CC0 / CC BY / CC BY-SA) だけを集め、100 か所中 54 か所に付けてある。作者名とライセンスはポップアップに出し、全 54 枚の一覧を /about に載せている。

写真を集めるときに引かなかったこと

スポット名で検索すると同名の別の場所が大量に混ざる。愛知県日進市の弁天池公園、岐阜県北方町の北方小学校、郡山市立行徳小学校などが実際に出てきた。そのため、座標による検索か専用カテゴリで裏を取り、最後に必ず目視で確認している。「AI に集めさせて終わりではないか」と聞かれたら、ここを答えればよい。

5-7 行政の応答 (F-7)

行政ユーザーは、投稿に公式コメントを付け、対応状況を 4 段階 (未対応・受付・対応中・対応済) で更新できる。対応状況を足したのは要件そのものではなく、設計の過程で加えたものである。理由は、参考にした市民報告サービスがいずれも対応状況を持っており、「報告したあとどうなったか」が見えないと、市民は 2 度目を投稿しないからである。

更新の権限は「含まれている項目ごと」に確かめる

更新は 1 つの経路 (PATCH /api/reports/:id) にまとめてあるが、要る権限は項目によって違う。

- ・対応状況を変えられるのは、担当市町村が一致する行政ユーザーだけ。市川市の職員が船橋市の投稿を閉じられてはいけない。
- ・本文を書き換えられるのは投稿者本人だけ。行政ユーザーであっても他人の本文には触れない。行政の言いはコメントとして残す。書き換えてしまうと、誰が書いたか分からなくなるためである。

もう 1 つ、実装上の細かい判断がある。更新のときに、カテゴリ固有の項目を丸ごと置き換えず、重ねている。そうしないと、浸水の投稿で本文を直すたびに、サーバーが焼き込んだ雨量が消えてしまう。

5-8 認証の二段構え (F-8)

要件は「Google アカウントでログイン」だが、そのまま作ると提出要件と正面衝突する。

提出要件	Google OAuth をそのまま入れた場合
審査員が <code>docker compose up</code> だけで動かせること	クライアント ID とシークレットが無いとログインできず、投稿機能が 1 つも試せない
認証キーが必須の外部サービスを使わない	Google の OAuth クライアントは必須キーそのもの
公開リポジトリにシークレットを置かない	<code>.env</code> を配ることもリポジトリに入れることもできない

そこで 鍵が両方そろっていれば Google モード、欠けていればデモモードという二段構えにした。判定は起動時にサーバー側で 1 回だけ行い、結果を画面へ渡す。クライアント側で鍵の有無を判定しない。

デモモードでも全機能が動くことが「要件」

投稿・写真のアップロード・コメント・行政操作まで、デモモードで全部試せる。ここを落とすと二段構えにした意味が無い。「できたらいい」ではなく要件として決めている。

5-9 セッションを、このデータベースに縛る

セッションは JWT で、DB にセッションのテーブルを持たない。テーブルとコードが 1 つ減るという利点があるが、そのままでは穴があった。

実測で見つかった穴 (2026-08-27)

別のデータベースで発行した「行政ユーザー」のトークンを、作り直した直後のデータベースへ持ち込んだところ、そのまま行政操作（対応状況の更新）が通った。トークンの中の利用者 ID はそのデータベースに存在すらしていなかった。権限の判定に使うロールと担当市町村も、トークン側の値だったからである。加えて、署名鍵が公開リポジトリに書かれた固定値だったので、中身を自分で組み立てて偽造することもできた。原因は 2 つ。利用者 ID がただの連番なので、データベースを作り直すと同じ番号が別人に割り当たること。そして Cookie はポート番号を区別しないので、同じ `localhost` の別インスタンス同士でも起きること。

対策は、データベースに 1 行だけ置いたインストール ID (起動時に UUID で決まる) を使った二重の縛りである。

何をするか	何を防ぐか
① 鍵が未設定なら、署名鍵をインストール ID から導く	公開の固定鍵による偽造。別インストールのトークンは復号すらできない
② 発行時に ID を焼き込み、毎回のリクエストで突き合わせる	人が鍵を固定して複数環境で共有していても、データベースが違えば通らない

要点は 鍵を 1 つも設定していない環境で成り立つことである。審査員が `docker compose up` だけで動かす前提を崩さずに、穴を塞いでいる。ID が読めないとき（データベースが落ちているとき）はセッションを通さない。例外は投げない。認証はアプリの入り口で毎回通るので、投げると画面ごと 500 になるからである。

副作用として、`docker compose down -v` でボリュームごと作り直すと ID も変わり、それ以前のログイン状態は無効になる。作り直したのだから投稿もユーザーも消えており、実害は無い。

5-10 行政ロールの PIN (公開運用のときだけ)

デモモードはロールを自己申告できる。審査員が行政側の機能まで試せるようにするための建付けで、手元で動かすぶんには正しい。ところがインターネットに公開すると、同じ建付けのまま 通りすがりの誰でも行政ロールで入り、対応状況を書き換えられる。

そこで環境変数を 1 つ設けた。未設定（審査員の既定）なら何も変わらず、設定した環境でだけ、行政ロールを選ぶときに PIN の一致を要求する。一般ユーザーのログインには一切影響しない。公開時に守りたいのは行政だけが持つ操作であって、投稿そのものではないからである。

PIN は短いので、総当たり対策を 2 段階構えで入れてある

① 失敗のたびに待たせる（連続失敗で指数的に伸ばし、8 秒で頭打ち）。② 連続 10 回失敗したら 60 秒は即座に断る。

① だけだと並列に投げられて意味が薄れるが、② を足すと並列でも 60 秒あたり 10 回までに絞れる。6 桁なら $10^6 \div 10 \times 60 \text{ 秒} \approx 69 \text{ 日}$ かかるので、発表期間の公開には十分である。比較そのものも、いったんハッシュに畳んでから定数時間で行っている（素直に比べると、応答時間から「何文字目まで合っているか」が漏れる）。

5-11 公開 URL をリクエストから決める

ログイン後のリダイレクト先も、Google のコールバック URL も、設定値ではなくリクエストが名乗ったホストから毎回導く。ホストは `X-Forwarded-Host` →（無ければ）`Host`、プロトコルは `X-Forwarded-Proto` である。

固定値をやめた理由 —— 実際に壊れた

以前は `AUTH_URL=http://localhost:3000` を固定で渡していた。この方式は「公開する住所を変えたら設定も直す」を人間が覚えている前提で、実際に破れた。公開ポートを 3100 に変えたら、ログイン後に開いていない 3000 番へ飛ばされた。

さらに、設定を外しただけでは直らなかった。`output: "standalone"` の Next.js は待ち受けアドレスからリクエスト URL を組むので、素のままだと `http://0.0.0.0:3000/...` が渡り、リダイレクト先が `0.0.0.0` になる。経路によって URL の決め方が違うのがこの問題の厄介なところで、サーバーアクション（サインイン・サインアウト）はヘッダーを見るので設定 1 つで足りるが、`/api/auth/*` のルートハンドラはリクエスト URL を見るので、自分で組み直す必要がある。

`X-Forwarded-Port` は見ない。Next.js がコンテナの中のポート（3000）を無条件に入れてしまうので、当てにすると 3100 番で開いた人を 3000 番へ飛ばす。ポート番号はホスト側に付いてくる。

これで、公開ポートを変えても Tailscale Funnel の HTTPS で公開しても、設定を 1 つも足さずに正しいリダイレクト先になる。どうしても固定したい運用のために、設定すればそちらが勝つ逃げ道も残してある。

5-12 書き出し（CSV / GeoJSON）

投稿は、画面で絞り込んだ条件そのまま CSV と GeoJSON に書き出せる。CHIZUBA はオープンデータの上に住民の投稿を重ねるサービスなので、集めたものもオープンデータとして返せて初めて一往復になる。行政が出したものを住民が使うだけの一方通行にしない、という考え方である。

決めたこと	なぜ
出すのは画面で誰でも読める項目と座標だけ	利用者 ID・メールアドレス・認証元の識別子は出さない。画面に出していないものは書き出しにも出さない
デモ投稿には印を付け、その雨量は書き出さない	デモの雨量は数字を置いてあるだけのダミーで、観測値ではない。観測値の顔をした列に入れると、受け取った人には見分けが付かない
CSV は UTF-8 の BOM 付き・CRLF	Excel が既定の文字コードで開いても日本語が化けないようにするため（RFC 4180 に BOM を足した形）
GeoJSON は RFC 7946 に従う	座標は【経度、緯度】の WGS84。座標系のメンバーは付けない（RFC 7946 は WGS84 に固定されている）
<code>= + - @</code> で始まるセルの先頭に <code>'</code> を足す	表計算ソフトが投稿の本文を数式として実行するのを止める。中身は変えない

5-13 入力 of 健全性

「悪意のある入力では壊れないか」は質疑で来やすい。入れてあるのは次のとおりである。

入り口	対策
写真	形式（JPEG / PNG / WebP）・1 枚 5 MB・3 枚・1 リクエスト 10 MB。申告された MIME だけでなく先頭バイトも検査する。拡張子と Content-Type は書き換えられるため
投稿の本文	タイトル 60 文字・本文 1000 文字。DB の CHECK 制約でも同じ上限を掛けている

入り口	対策
座標	千葉県範囲に収まらない座標は受け付けない (DB の CHECK 制約でも縛る)
所属市町村・雨量・公式フラグ	クライアントから受け取らない。サーバーが決める
カテゴリ固有の項目	定義に無いキーは捨てる。実測でも、デモ投稿の印を偽って送っても付かないことを確認済み
SQL	すべてプレースホルダで渡す。文字列を組み立てて SQL に埋め込んでいる箇所は無い
キーワード検索	部分一致のワイルドカード (% _) を打ち消す。利用者が % を打っても「全部に当たる」ことはない
CSV の書き出し	数式として実行されうるセルを無効化する (5-12)
行政ロール (公開時)	PIN + 総当たり対策 (5-10)

正直に言うておくこと

投稿そのものの悪用対策——荒らし・なりすまし・大量投稿——はハッカソンのスコープ外として意図的に入っていない。聞かれたら「入力 of 健全性は入れましたが、投稿数のレート制限や通報の仕組みは未実装です。実運用なら、まず 1 人あたりの投稿数の制限と、行政側の非表示操作から足します」と答える。「対策済みです」と言い切らないことが大事である。

5-14 デモデータの扱い

初回起動時に、デモの投稿を 22 件 (危険箇所 7・浸水 5・観光おすすめ 10) 入れてある。空の地図では何も伝わらないためだが、デモを実データに見せない建付けを入れてある。

- 一覧と詳細に デモのバッジを出す。利用者は投稿からこのバッジを詐称できない
- 浸水のデモに入れてある雨量は 気象庁の 出典を出さず、「デモ用のダミー値」とだけ表示する。過去の実測値は気象庁の配信からは取れないので、観測値のふりをさせない
- 写真はウィキメディア・コモンズの再利用可能なものだけ。防災の 7 枚は市外の参考写真であり、市川市で実際に起きた被害の写真ではない旨を /about に明記している

浸水のデモ 5 件は 3 つの「雨のあった日」に散らしてある。全部が同じ日に固まっていると、期間で絞る操作を試せないためである。

想定質疑と模範回答

答えは 3～5 文にしてある。質疑は 1 問あたり 30 秒程度で、それ以上話すと次の質問に行けない。最初の 1 文で結論を言い、残りで根拠を 1 つか 2 つ足すと収まる。分からないことを聞かれたら、「そこは未実装です」と言い切ってから、代わりに何を入れたかを言う。取り繕うと次の質問で崩れる。

仕組みと技術選定について

Q1 F-4 の注意案内は「予測」ではないのですか

予測ではありません。出しているのは事実 2 つだけで、「過去にこの地域で浸水の報告が N 件あります」と「気象庁の予報では今後 24 時間の降水確率が最大 X% です」を並べているだけです。独自のモデルも推定も持っていません。気象業務法が気象庁以外の予報を許可制にしているので、「浸水するでしょう」に類する表現はコードにも画面にも入れていません。画面にも「浸水の予測ではありません」と明記しています。

Q2 なぜ Google マップを使わないのですか

API キーと課金アカウントが必須で、審査員の環境で地図が出せなくなるからです。加えて、このアプリの主役は国土地理院のタイルと国土交通省のハザードタイルを重ねることで、任意の XYZ タイルをそのままレイヤーにできる MapLibre のほうが素直です。MapLibre は BSD ライセンスのオープンソースなので、利用規約の変更で急に使えなくなる心配もありません。

Q3 PostGIS を使えばもっと簡単だったのでは

必要な地理演算が矩形の範囲検索と 2 点間の距離だけなので、使いませんでした。どちらも緯度経度の数値カラムと Haversine 式で足りる。提出要件が「ベースイメージは可能な限り Docker 公式イメージ」と定めていて、PostGIS のイメージは公式ではないという理由もあります。PostGIS が要るのはポリゴンとの交差判定を DB でやりたいときですが、ハザードマップはタイル画像として重ねるだけなので、その必要がありません。

Q4 データベースの設計を説明してください

テーブルは 6 つです。市町村マスタ、ユーザー、投稿、投稿の写真、投稿へのコメント、それにインストール ID を 1 行だけ持つテーブルです。投稿は危険箇所・浸水・観光おすすめを 1 テーブルにまとめていて、違いはカテゴリの値と、カテゴリ固有の項目を入れる JSON の列だけです。位置は緯度経度の数値カラムで、値域は DB の CHECK 制約でも縛っています。写真は実体をボリュームに置き、DB にはファイル名だけを記録しています。

Q5 なぜ投稿を 1 つのテーブルにまとめたのですか

実装量が 3 倍にならないようにするためです。3 つに分けると API も 3 本、フォームも 3 つ、地図のレイヤーも 3 系統になり、一覧を出すたびに 3 本の問い合わせを束ねることになります。共通する部分——位置・写真・本文・投稿者・対応状況——が全体の 9 割を占めていました。そのうえで、地図で最初から表示する組と、ピンの色はカテゴリごとに分けてあります。

Q6 スケールしますか。投稿が何万件になったら

今の作りは 1 市あたり数百件を想定した規模です。地図の表示範囲・カテゴリ・期間で絞る列には索引を張っているので、数万件までは今のままで動くと思います。それ以上になるなら、まずベクタタイルへの切り替えと、ズームに応じた点の集約が要ります。今は「動くこと」を優先し、先回りの最適化はしていません。

Q7 オフラインでも使えますか

使えません。背景地図とハザードマップのタイルを外部から取るので、通信が無いと地図が出ません。ただし外部サービスが落ちてでも止まらないにはしてあります。施設のデータはアプリに同梱しており、投稿は自前のデータベースにあるので表示できます。経路サービスが落ちたときは直線距離からの見積もりに切り替え、見積もりであることを画面に出します。

Q8 テストは書きましたか

自動テストは書いていません。代わりに、各段階の完了条件として 型検査・`docker compose up` での実操作・提出アーカイブの起動確認・秘匿情報のスキャンの 4 つを毎回通しています。外部サービスが落ちたときの挙動は、通信を遮断した状態で実際に投稿して確かめました。期間が 2 週間強で、テストを書く時間を機能の動作確認に振った、というのが正直なところです。

Q9 どこまで自分たちで作ったのですか

外部から持ってきているのは 地図タイル・ハザードタイル・経路計算・気象データで、いずれも公的機関か非営利団体が公開しているものです。それらをどう組み合わせ、何を投稿の形にし、どこで線を引くかは自分たちで決めました。実装には AI コーディングツールを使っていますが、設計判断——投稿を 1 テーブルにする、PostGIS を使わない、予報表現を入れない——は理由まで含めてドキュメントに残してあります。

データと権利について

Q10 写真の権利はどうしていますか

デモに付けた写真も景観スポットの写真も、ウィキメディア・コモンズで再利用が許されているもの (CC0 / CC BY / CC BY-SA) だけを使っています。作者名とライセンスは画面に出していて、全 54 枚の一覧を `/about` に載せています。スポット名で検索すると 同名の別の場所が大量に混ざるので、座標か専用カテゴリで裏を取ったうえで、最後は目視で確認しました。防災のデモに付けた 7 枚は 市外の参考写真で、市川市の被害写真ではない旨も明記しています。

Q11 オープンデータの出典はどう扱っていますか

出典の文言は コードの 1 箇所にまとめてあり、地図の隅と `/about` の両方が同じものを読んでいます。市川市のオープンデータは CC BY 4.0、ハザードマップと気象庁のデータは 公共データ利用規約 (第 1.0 版) です。気象庁のデータは 加工した旨も書くことが求められているので、「最寄りの観測所の抽出と距離の算出を行って表示している」と明記しています。

Q12 個人情報はどう扱っていますか

メールアドレスと認証元の識別子は、画面にも API の応答にも一切出しません。外へ出すのは表示名とロールだけです。CSV や GeoJSON への書き出しでも同じで、画面に出していないものは書き出しにも出しません。Google ログインで受け取ったメールアドレスは、行政ロールを付けるかどうかの判定にだけ使っています。

Q13 投稿された雨量は、本当にその場所の値ですか

いいえ、最寄りのアメダスの値です。市川市にはアメダスが無く、最寄り船橋で約 10 km あります。そのため 観測所名と距離を必ず画面に併記しています。「船橋アメダス (約 10.2 km)」という形です。20 km より遠い場合は、その地点の雨量として使わずに空欄にします。

運用と実用性について

Q14 「千葉県全域」と言いますが、データは市川市だけでは

そのとおりで、デモデータは市川市だけです。ただし 骨格は最初から全域対応で作ってあります。市町村は 5 桁のコードで識別し、地図の初期表示の座標もズームも範囲も マスタのテーブルから引いていて、コードに座標を書いていません。市を 1 つ増やす作業は「マスタに 1 行足して、その市の CSV を GeoJSON にする」だけで、コードの変更を伴いません。

Q15 デモモードだと誰でも行政ロールを名乗れるのでは

手元で動かすぶんには、そのとおりです。審査員が投稿から行政の対応まで通して試せるように、あえてそうしてあります。ただし インターネットに公開するときは、行政ロールに PIN を要求する仕組みを入れてあります。環境変数を 1 つ設定した環境でだけ有効になり、未設定なら何も変わりません。総当たり対策として、失敗ごとの待機と連続失敗のロックも入れてあります。

Q16 荒らしや大量投稿への対策はありますか

未実装です。入れているのは入力健全性——写真の形式とサイズ、文字数の上限、座標の範囲、所属市町村をサーバーが決めること——までです。投稿数のレート制限も、通報の仕組みも、行政による非表示操作もありません。実運用に載せるなら、まず 1 人あたりの投稿数の制限と、行政側の非表示操作から足します。

Q17 既存のハザードマップポータルと何が違うのですか

ハザードマップポータルは「想定」を見せるサイトで、そこに「今どうなっているか」は載りません。CHIZUBA は同じタイルを重ねたうえに、住民が投稿した浸水と危険箇所を並べています。想定と実績を同じ地図で見られること、そして集めた実績を CSV と GeoJSON で返せることが違いです。

Q18 市民報告型のサービスは既にありますが、違いは何ですか

違いは 3 つあります。第 1 に、防災と観光を同じ地図・同じ投稿の仕組みで扱っていること。第 2 に、オープンデータの上に投稿を重ねていることで、避難場所や景観スポットといった既存のデータと、住民の投稿が同じ画面に並びます。第 3 に、集めた投稿をオープンデータとして書き出せることです。行政が出したものを住民が使うだけの一方通行にしない、というのが設計の軸です。

Q19 行政が本当に運用できますか

行政側に専用の管理画面を作っていないのが、この質問への答えです。職員も住民と同じ地図画面を使い、ログインすると出せる操作が増えるだけにしてあります。操作は **対応状況を 4 段階から選ぶこと**と、**コメントを書くこと**の 2 つだけです。他市の投稿は触れないよう、担当市町村が一致するかをサーバー側で確かめています。ただし職員アカウントの管理そのものはスコープ外で、今は環境変数にメールアドレスを書く形にとどめています。

Q20 災害のときに本当に動きますか

そこは正直に言うと、今の構成では保証できません。自宅の Mac 1 台で動かしていて、冗長化もしていないので、停電や回線断があれば止まります。設計として入れてあるのは「**外部サービスが落ちても地図と投稿は出る**」ところまでで、施設のデータをアプリに同梱し、経路は直線距離の見積もりに切り替わるようにしてあります。実運用に載せるなら、まずホスティングの冗長化から必要です。

Q21 実装で一番苦労したところはどこですか

セッションの穴です。別のデータベースで発行したトークンを持ち込むと、そのデータベースに存在しない利用者として **行政操作が通ってしまう**ことが分かりました。Cookie がポート番号を区別しないので、同じ `localhost` でも起きます。データベースにインストール ID を 1 行置いて、署名鍵をそこから導き、トークンにも焼き込んで毎回突き合わせる二重の縛りで塞ぎました。鍵を 1 つも設定していない環境でも成り立つことが条件でした。

Q22 これからどう発展させますか

近いところでは、投稿が溜まったときの使い道です。どの地点がどの雨量で冠水したかというデータセットができるので、それを行政や研究者が使える形で渡すのが筋だと考えています。CHIZUBA 自身が予測を出すつもりはありません。そこは気象庁と自治体の仕事で、私たちがやるべきなのは **現場の記録を集めて、使える形**にして返すことです。

用語集

質疑で出てきそうな語を、1 ～ 2 行で言えるようにまとめてある。「それは何ですか」と聞き返されたときに、ここの言い方をそのまま使えばよい。

■ ウェブとサーバー

App Router Next.js の画面とサーバーの API を、同じディレクトリ構成の中に並べて書ける仕組み。

サーバーコンポーネント サーバー側でだけ実行される画面の部品。ブラウザへは結果の HTML だけが届くので、鍵や DB の接続情報がブラウザに漏れない。

ルートハンドラ /api/... のような URL に対して、サーバー側で動く関数。CHIZUBA では投稿・気象・経路・写真の 4 系統がある。

多重定義 (multipart) 写真のようなファイルと文字を、1 回の送信にまとめて詰める形式。投稿はこの形で送っている。

User-Agent 「どのソフトが要求したか」を名乗るヘッダー。OSRM の利用規約が正しく名乗ることを求めている。

レート制限 単位時間あたりの要求回数の上限。OSRM は 1 秒 1 回で、CHIZUBA はサーバー側の順番待ちで守っている。

キャッシュ 取ってきた結果をしばらく手元に置いて、同じ要求で外部へ行かないようにすること。寿命 (TTL) を決めて持つ。

タイムアウト 応答を待つ上限時間。超えたら諦める。外部が落ちたときに画面が固まらないために要る。

リバースプロキシ 利用者と本体のあいだに立って要求を転送するもの。元のホスト名を X-Forwarded-Host など で伝える。

TLS 終端 暗号化された通信 (HTTPS) を途中で復号すること。Tailscale Funnel はここで復号して、中は素の HTTP で本体へ渡す。

■ 認証とセキュリティ

OAuth 「Google のアカウントで入る」を実現する規格。パスワードを渡さずに、本人であることだけを別のサービスに証明してもらう。

JWT 署名付きの小さなデータ。中身は誰でも読めるが、署名があるので書き換えると分かる。CHIZUBA のセッションはこの形。

Cookie ブラウザがサーバーから預かって毎回送り返す小さなデータ。ポート番号を区別しないという性質が、5-9 節の穴の原因になった。

署名鍵 JWT の署名を作る秘密の値。これが漏れるとトークンを偽造できる。CHIZUBA は DB のインストール ID から導いている。

ハッシュ (SHA-256) 任意の長さのデータを、決まった長さの値に潰す一方方向の計算。元に戻せない。

タイミング攻撃 比較にかかる時間の差から秘密を推測する攻撃。素直に文字列を比べると「何文字目まで合っているか」が漏れる。

プレースホルダ SQL に値を埋め込むのではなく、別枠で渡す書き方。SQL インジェクションを防ぐ基本で、CHIZUBA は全箇所これ。

マジックバイト ファイルの先頭にある、形式を示す決まったバイト列。申告された形式が嘘でも、ここを見れば分かる。

CSV インジェクション CSV のセルが表計算ソフトで数式として実行される問題。先頭に ' を足して無効化する。

■ 地図とデータ

XYZ タイル 地図を正方形の画像に切り分けて、ズーム・列・行の 3 つの番号で取り出す方式。国土地理院もハザードマップも、この形で配信している。

ラスタタイル 画像として配られるタイル。重ねるだけで表示できるが、中の値 (浸水深など) をプログラムから読むことはできない。

ベクタタイル 点や線の座標として配られるタイル。拡大しても綺麗だが、作るのに手間がかかる。CHIZUBA では使っていない。

GeoJSON 位置付きのデータを表す JSON の標準形式. CHIZUBA は 静的データも投稿も同じこの形で扱う.
FeatureCollection GeoJSON で「地物の集まり」を表す入れ物. 投稿の一覧はこの形で返る.
WGS84 GPS で使われている世界共通の座標系. GeoJSON はこれに固定されている.
bbox (矩形) 「西・南・東・北」の 4 つの数で表す長方形の範囲. 地図の表示範囲で投稿を絞るのに使う.
Haversine 式 球面上の 2 点間の距離を求める式. アメダスまでの距離も, 最寄りの避難場所を探すのもこれ.
JIS X 0402 市区町村を表す 5 桁のコードの規格. 市川市は 12203. CHIZUBA は全域対応の鍵としてこれを使う.
Okabe-Ito 色覚多様性に配慮した配色として国内で実績のある 8 色の組. 地図の点も凡例もこれで塗っている.
NFKC 正規化 全角と半角, 大文字と小文字などの見た目の違いをそろえる変換. 検索が「アンダーパス」でも当たるのはこれのおかげ.

■ Docker とデータベース

イメージ/コンテナ イメージは実行に必要なものを固めた型で, コンテナはそれを動かした実体.
Docker 公式イメージ (DOI) Docker 社が品質を確認して配布しているイメージ. 提出要件が「可能な限りこれを使う」と定めている.
multi-stage ビルド 「依存の取得」「ビルド」「実行」を別の段に分ける書き方. 実行イメージに開発用の道具を持ち込まないためにする.
healthcheck コンテナが「使える状態か」を定期的に確かめる仕組み. db は pg_isready で確かめている.
depends_on: service_healthy 相手の healthcheck が通るまで自分を起動しない指定. DB の準備が済む前に web が繋ぎに行くのを防ぐ.
ボリューム コンテナを作り直しても消えない保存領域. CHIZUBA は DB のデータと投稿写真の 2 つを持っている.
ORM SQL の代わりにプログラムの言葉で DB を操作する道具. CHIZUBA は入れていない (依存を増やさず, ビルドが壊れる余地も作らないため).
トランザクション 複数の書き込みを「全部成功か, 全部無かったことにするか」のどちらかにまとめる仕組み. 投稿の作成で使っている.
インデックス (索引) 特定の列で速く探せるようにする仕組み. CHIZUBA は緯度・経度・市町村・カテゴリ・投稿日時に張っている.
CHECK 制約 列に入れてよい値を DB 側で縛る仕組み. 文字数や座標の範囲を, プログラムと DB の両方で確かめている.
jsonb PostgreSQL が JSON を効率よく保存する型. カテゴリ固有の項目をここに入れている.

■ 気象と法令

アメダス 気象庁の地域気象観測システム. 観測所は全国に約 1,300 地点あり, 10 分ごとに値が配信される. すべての地点が雨量を測っているわけではない (風だけの地点もある).
予報区 天気予報を出す区域の単位. 市町村より粗く, 千葉県は北西部・北東部・南部の 3 区分.
降水確率 その時間帯に 1 mm 以上の雨が降る確率. 気象庁が 6 時間ごとに 10% 刻みで出す.
気象業務法 気象庁以外が予報を業として出すことを許可制にしている法律. F-4 が事実の提示に留まっている理由.
公共データ利用規約 (第 1.0 版) 国の機関のデータを出典明記で使える規約. ハザードマップと気象庁のデータがこれ.
CC BY 4.0 出典を明記すれば自由に使えるクリエイティブ・コモンズのライセンス. 市川市のオープンデータがこれ.
ODbL OpenStreetMap のデータのライセンス. 徒歩経路の道路データがこれにあたる.

数字の早見表

このページだけは直前に見ておく。質疑で数字を間違えると、そこから先の話がすべて疑われる。分からない数字は「正確な数は手元の資料にあります」と言えばよく、推測で答えない。

データの件数

項目	数	項目	数
指定緊急避難場所	123	デモ投稿（合計）	22
AED 設置箇所	304	うち危険箇所	7
子育て施設	388	うち浸水	5
景観 100 選	100	うち観光おすすめ	10
うち写真つき	54	デモ投稿の写真	17

構成と規模

項目	数	項目	数
コンテナ	2	機能（F-1 ～ F-8）	8
名前つきボリューム	2	投稿のカテゴリ	3
データベースのテーブル	6	対応状況の段階	4
ハザードのレイヤー	3	ユーザーのロール	3
TypeScript の行数（ app/src ）	約 10,600	db/init の SQL	約 370 行

上限としきい値

項目	値	項目	値
写真 1 枚	5 MB	タイトル	60 字
写真の枚数	3 枚	本文	1000 字
1 リクエスト	10 MB	コメント	500 字
一覧の既定件数	500 件	表示名	30 字
一覧の上限件数	1000 件	検索語	40 字

外部サービスとの約束

項目	値
注意案内を出す降水確率	30%（今後 24 時間の最大値）
アメダスの最大距離	20 km. 市川市の最寄り船橋で約 10 km
キャッシュの寿命	アメダス実況 10 分／府県予報 30 分／観測所一覧・地域区分 24 時間
気象庁のタイムアウト	5 秒
OSRM の間隔とタイムアウト	1.1 秒に 1 回／8 秒
ハザードタイルのズーム	配信は 2 ～ 17. 6 より引いたら描かない
ハザードの不透明度	既定 0.6（0.1 ～ 1.0 の範囲で変えられる）
PIN の総当たり対策	失敗ごとに最大 8 秒待機／連続 10 回で 60 秒ロック

覚えておく固有の値

項目	値
市川市の市町村コード	12203 (JIS X 0402)
公開ポート	3000 (CHIZUBA_PORT で変えられる。変えても他に直す設定は無い)
ベースイメージ	node:22-slim / postgres:17-alpine (どちらも Docker 公式イメージ)
コード提出の締切	2026-09-09 (水) 17:00
プレゼン資料の締切	2026-09-16 (水) 11:50 (発表 10 分 + 質疑 3 分)

質疑で困ったときの 3 つの型

- ① 未実装を聞かれたら——「そこは未実装です。代わりに〇〇までは入れました。実運用に載せるなら、まず××から足します」。
- ② 精度や正確さを聞かれたら——「その値は〇〇由来で、画面にも出典と条件を出しています」。断定しないことが誠実さになる。
- ③ 知らないことを聞かれたら——「そこは確認していません」と言い切る。推測で答えると、次の質問で必ず崩れる。

もっと詳しく知りたくなったら

<code>docs/presentation/chizuba-overview.progfocus.md</code>	本書の図の正本。prog-focus に取り込むと、ノード 38・接続 34 の図として開ける
<code>docs/design/requirements.md</code>	何を作るかの正本。機能 F-1 ~ F-8・データモデル・認証・実装順序
<code>docs/design/interfaces.md</code>	つなが目の正本。API の形式と、異常時の約束
<code>.agent/architecture.md</code>	システム構成と設計判断の一覧
<code>app/README.md</code>	動かし方と既知の制約
<code>課題/2026-09-09_CODIHA2026_提出要件.md</code>	提出要件の正本。締切・0 点条件・審査基準

この PDF は `chizuba-tech-explainer.typ` から `typst compile chizuba-tech-explainer.typ` で作り直せる。